Docker

MLOps

Docker

ПО для автоматизации развёртывания и управления приложениями в средах с поддержкой контейнеризации

- Контейнеризация - метод виртуализации, при котором ядро ОС поддерживает несколько изолированных экземпляров (контейнеров) пространств пользователя вместо одного.
- Контейнер - стандартная рабочая единица.
- Контейнер имеет одинаковое поведение в любом окружение.
- Большинство ОС поддерживают контейнеризацию.

Преимущества контейнеров

- Возможность задавать детали окружения, так как необходимо тестировать работу на том же окружении, на котором оно будет запущено.
- Возможность "поделиться" рабочим окружением в команде/компании.
- Даёт возможность легко переключаться между подпроектами (более зрелые подпроекты могут работать по более старых версиях).

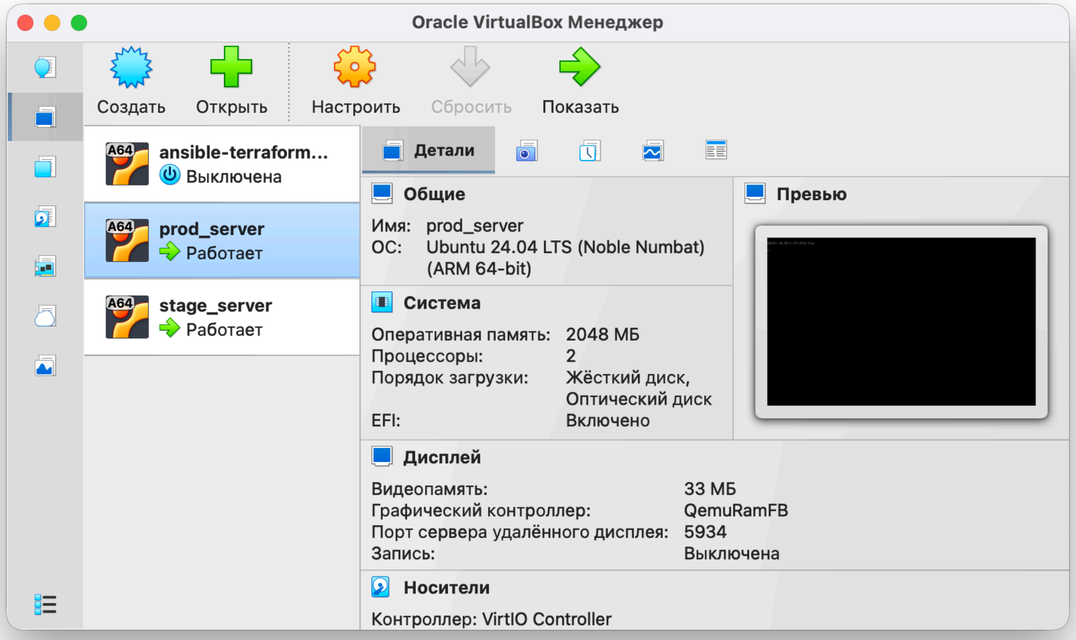
Виртуальная машина

- Программная и/или аппаратная система, эмулирующая аппаратное обеспечение компьютера и исполняющая программы для guest-платформы (guest - гостевая платформа) на host-платформе (host - хост-платформа) или виртуализирующая некоторую платформу и создающая на ней среды, изолирующие друг от друга программы и даже операционные системы.

Преимущества и недостатки виртуальной машины

- Защита через изоляцию
- Интерактивное развитие (более удобное и гибкое)

- Скорость цикла разработки
- Затраты на хранилище



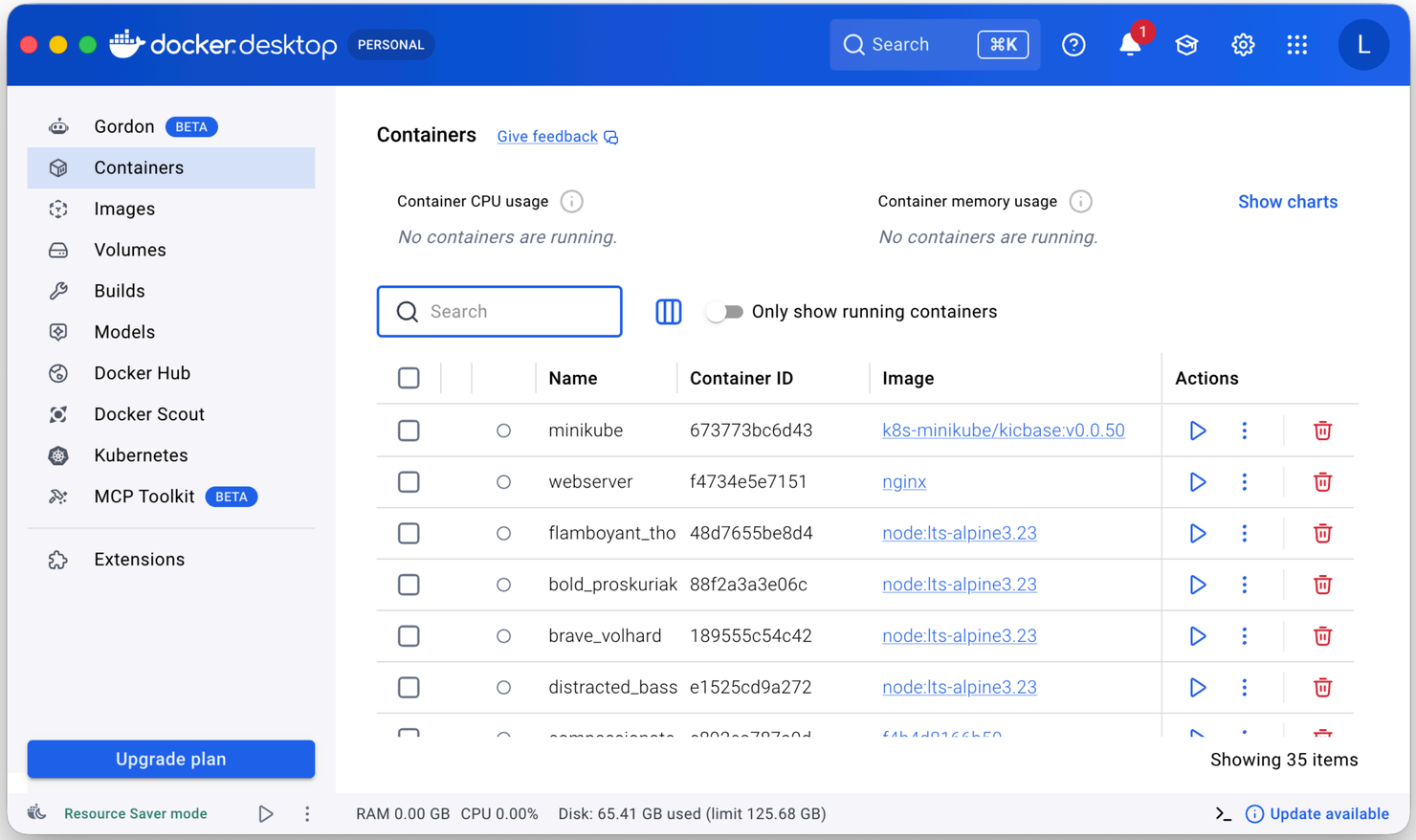
Docker

Преимущества и недостатки

- Скорость итераций разработки
- Надёжная экосистема

(благодаря наличию репозиториев)

- Уязвимость общего узла

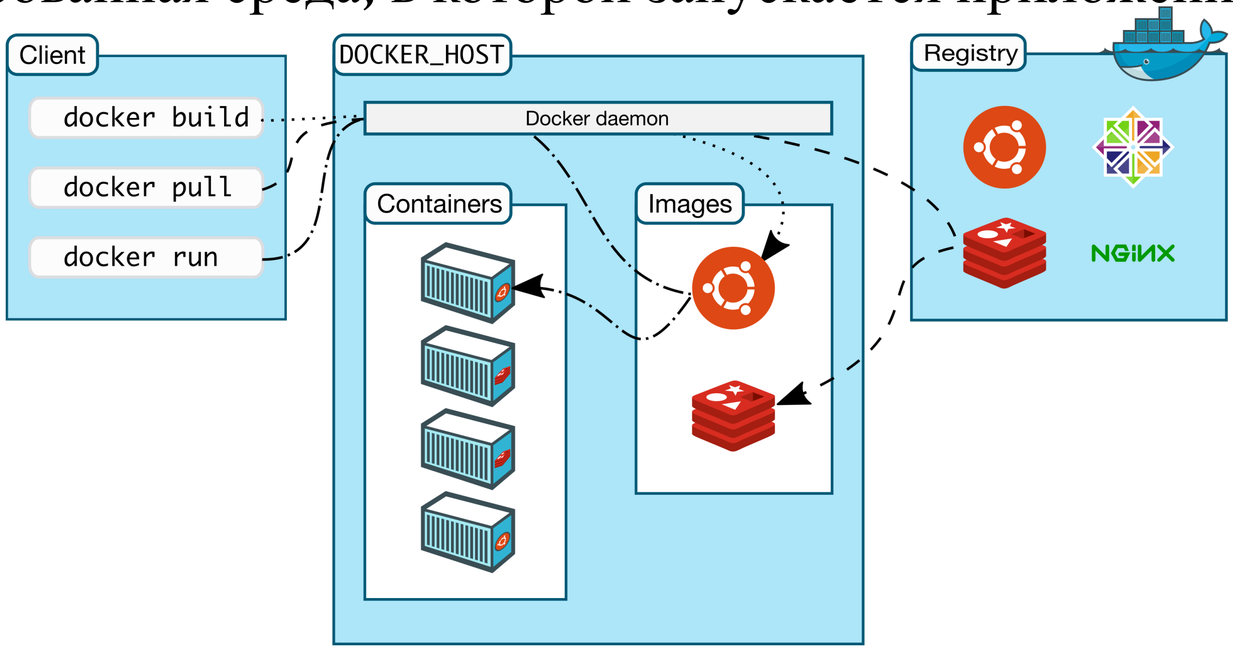


Основные компоненты Docker

- Docker Engine - ядро Docker (которое отвечает за создание, запуск и управление контейнерами)
- Docker Desktop - UI для Docker
- Docker Compose - позволяет управлять более сложными мультиконтейнерными приложениями
- Docker Hub - позволяет размещать образы не локально, а удаленно
- Docker Daemon - это серверное приложение, которое отвечает за выполнение всех задач Docker, работает в фоновом режиме на системе-хосте и управляет объектами Docker: образами, контейнерами, сетями и томами.

Определения Docker

- Docker образ - шаблон для контейнера, который состоит из исходного кода, всех нужных зависимостей
- Контейнер - изолированная среда, в которой запускается приложение из образа (рабочая лошадка)



Пример Dockerfile

Нужен для упаковки в Docker образ

- docker build . -tag=nodeapp:latest
- docker run -p 4200:4200 nodeapp:latest

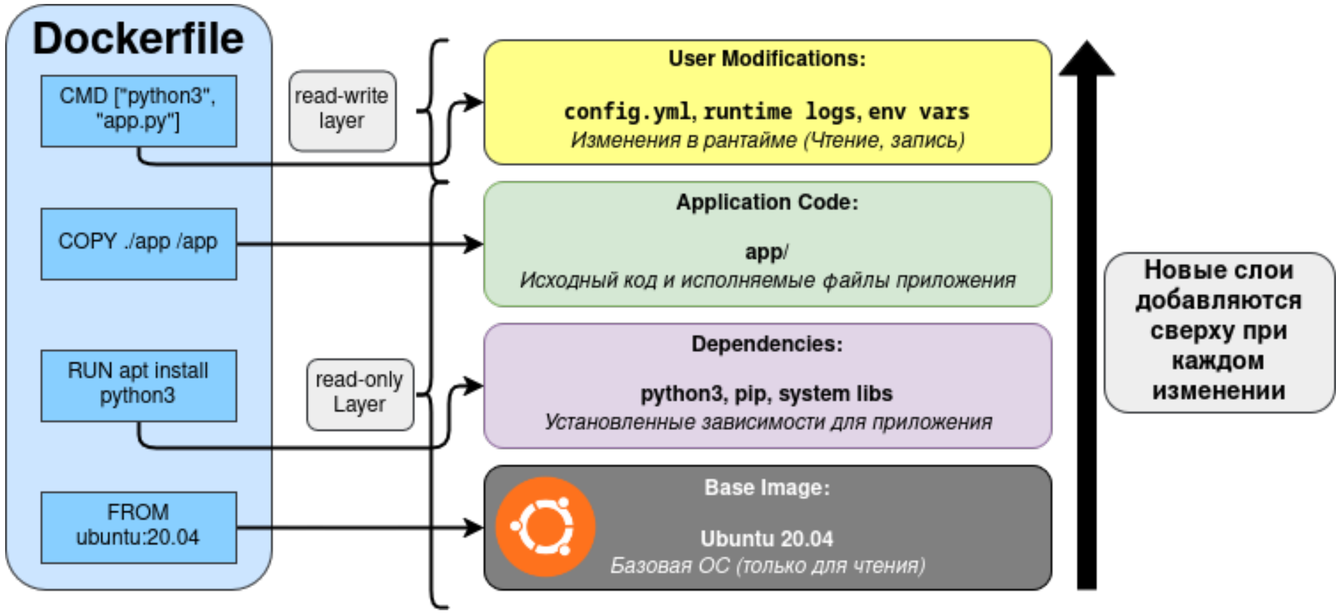
```
FROM node as build
WORKDIR /app
COPY. /app
RUN npm install
EXPOSE 4200
ENTRYPOINT ["npm", «start"]
```

- Нельзя добавить изменения в образ, его надо пересобрать

Слои образа

Слои могут быть кэшированы (отключить —no-cache)

- RUN, COPY, ADD - создают новые слои.
- ENV, LABEL, EXPOSE - добавляют метаданные, но не создают слои.



Утилитные контейнеры

- Контейнеры, которые представляют собой только окружение (Environment).

- docker pull rust
- docker run —name rust_name_container -it -d rust
- docker exec rust_name_container rustc —version

```
leomag@MacBook-Pro-Ilya-Spb ~ % docker exec rust_name_container rustc --version
rustc 1.94.1 (e408947bf 2026-03-25)
leomag@MacBook-Pro-Ilya-Spb ~ %
```

Команды docker

- docker ps - вернуть список активных контейнеров
- docker ps -a - вернуть список все контейнеров
- docker kill id_container - убить контейнер
- docker stop id_container - остановить контейнер
- docker stop $(docker ps -a -q) - остановить все контейнеры
- docker rm id_container - удалить контейнер
- docker container prune - удалить все неактивные контайнеры

Команды docker

- docker run -d -p name_port:name_port —name name_container user_name:version - запустить образ в контайнере на порту в неблокирубщем режиме терминала с заданным именем контейнера
- docker attach id_container - позволяет подключиться к уже запущенному контейнеру и взаимодействовать с его процессами напрямую через командную строку
- docker logs id_container -f - смотреть логи
- docker logs id_container —tail 100 - смотреть определённое количество логов

Команды docker

- docker image ls - вернуть docker образы
- docker inspect image_name - вернуть низкоуровневой информации об объектах в образе в JSON формате
- docker rmi image_id - удалить образ
- docker image prune - удалить неиспользуемые образы

Публикация образов в реестре Docker Hub

- docker build . -tag=nodeapp:latest leomag/nodeapp:latest
- docker push leomag/nodeapp:latest

- docker pull leomag/nodeapp:latest
- docker run -p 4200:4200 leomag/nodeapp (сразу pull из реестра и запустит в контейнере)

Пример multi-stage Dockerfile

```
## Build stage
FROM gradle:jdk21-ubi9 AS builder
LABEL maintainer="leomag.isaev@gmail.com"
WORKDIR /app
COPY build.gradle.kts settings.gradle.kts ./
COPY gradle ./gradle
RUN gradle --no-daemon dependencies
COPY src ./src
RUN gradle bootJar --no-daemon

## Package stage
FROM eclipse-temurin:21-jre-jammy
RUN adduser --disabled-password --gecos '' app
USER app
WORKDIR /app
COPY --from=builder /app/build/libs/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Пример multi-stage Dockerfile

```
# builder
FROM python:3.11-slim AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y gcc && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

# runtime
FROM python:3.11-slim
RUN groupadd -r appuser && useradd -r -g appuser appuser
WORKDIR /app
COPY --from=builder /root/.local /home/appuser/.local
COPY ./app /app
RUN chown -R appuser:appuser /app
USER appuser
ENV PATH=/home/appuser/.local/bin:$PATH
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Работа с переменными

- docker run -rm -p 4200:4200 -e ‘PATH’=/home/user —name name_container

- docker run -rm -p 4200:4200 —env-file .env —name name_container (когда есть файл .env)

- docker run -rm - удаляет автоматически контейнер после его установки

Работа с данными

Тип данных	Исходный код + Окружение	Временные данные	Постоянные данные
Этап получения данных	Сборка приложения	Работа приложения	Работа приложения
Тип данных	Неизменяемы	Изменяемы	Изменяемы
Место хранения	Docker образ	Docker контейнер	"Постоянное хранилище"

Работа с данными

- Каждый контейнер изолирован от файловой системы другого контейнера.
- Удаляя контейнер удаляются все его данные.
- Том (volume) - механизм постоянного хранения данных.
- Когда контейнера удален, то том нет.
- Доступ к тому есть на чтение и запись.

Работа с томами

- docker run -p 4200:4200 -rm —name node -e storage.location=‘/tmp/storage.txt’ nodeapp:latest

- docker volume ls - вернуть все тома
- docker rm volume_name - удалить том
- docker volume inspect volume_name - вернуть информацию о томе

Сеть в Docker

- Bridge - сеть по умолчанию, создаёт виртуальный сетевой мост, через который контейнеры могут взаимодействовать друг с другом и с хостом.
- Host - позволяет контейнерам работать напрямую с сетевыми интерфейсами хоста.
- Overlay (перекрывающая сеть) - позволяет контейнерам, запущенным на разных хостах, взаимодействовать друг с другом, создавая виртуальную сеть поверх физической сети.
- Macvlan - позволяет контейнерам иметь уникальные MAC-адреса, что делает их видимыми в физической сети как отдельные устройства.
- None - контейнер не имеет никакого сетевого интерфейса.

Сеть в Docker

Пример обращение к БД Mongo в контейнере:
ENV mongo.host 'host.docker.internal'
ENV mongo.port '27017'
ENV mongo database ‘mongo'

host.docker.internal - специальное DNS-имя, оно преобразуется в внутренний IP-адрес хост-машины. Это позволяет контейнерам общаться с любыми сервисами, которые доступны через хост - независимо от того, работают они нативно на машине или в другом контейнере.

Создать свою сеть

- docker network ls - вернуть все сети
- docker network create name_network - создать свою сеть (по умолчанию типа сети bridge)
- docker run -p 27017:27017 --name mongo_container -d --network my_network mongo
- docker network create —driver type_network name_network - создать свою сеть с указанным типом

Docker Compose

инструмент для запуска и управления приложениями, состоящими из нескольких Docker-контейнеров, позволяет описать все компоненты ПО в одном конфигурационном файле и запустить их одной командой

- version
- services
- image
- container_name
- build
- environment
- ports
- depends on

Пример docker-compose.yml

```yml
services:
  db:
    image: postgres:latest
    container_name: postgres_db
    environment:
      POSTGRES_USER: your_user
      POSTGRES_PASSWORD: your_password
      POSTGRES_DB: your_db_name
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data/pgdata

  fastapi_app:
    image: fastapi_image:latest
    ports:
      - "5000:5000"
    depends_on:
      - db
```

Команды docker-compose

- docker-compose up - запускает все сервисы, описанные в файле docker-compose.yml (при необходимости создаёт соответствующие контейнеры, сети и тома)
- docker-compose up -d - запускает контейнеры в фоновом режиме
- docker-compose up -d name_service - запускает только контейнер с указанным сервисом
- docker-compose down - останавливает и удаляет контейнеры, сети и тома, созданные с помощью команды up
- docker-compose stop - приостанавливает работу всех сервисов, описанных в docker-compose.yml
- docker-compose build - собирает или пересобирает образы на основе инструкций в docker-compose.yml и Dockerfile